

SSH Tunneling for Modbus/TCP Protocol

Project Team Members:

Pavai Vendhan Ganesan – 22408095

Nandukrishna Menon – 12640669

Joyal Joshy – 12630286

A project report presented for the module
**Selected Topics of Embedded Software
Development I**



Embedded Security – MASTER Applied Computer Science
Summer Term 2026

Source Code Repository:

<https://github.com/pavaivendhan/ESD-Modbus-SSH-Tunnel>

Abstract

The reliance of modern critical infrastructure on legacy industrial communication protocols presents a severe security risk. Modbus/TCP, a pervasive protocol utilized in Supervisory Control and Data Acquisition (SCADA) systems and Programmable Logic Controllers (PLCs), was designed long before cyber-physical attacks became a common threat. It inherently lacks any mechanisms for authentication, encryption, or cryptographic integrity checking. Because of this, industrial networks that use Modbus/TCP are vulnerable to eavesdropping, Man-in-the-Middle (MITM) attacks, and unauthorized data injection, potentially leading to catastrophic physical damage.

In this report, we break down the theoretical vulnerabilities of the Modbus/TCP protocol. We then demonstrate these flaws in practice by capturing network traffic with Wireshark. To fix this security flaw without replacing existing legacy hardware, this project proposes and implements a cryptographic mitigation strategy utilizing Secure Shell (SSH) tunneling. By wrapping the Modbus traffic inside an AES-encrypted SSH tunnel, we managed to add confidentiality, integrity, and authentication to a system that wasn't built for it.

We also look into whether it is actually feasible to deploy this setup on embedded systems with limited resources. We ran benchmarks to see exactly how much CPU and latency overhead the encryption was adding. Our tests clearly show that SSH tunneling provides an effective and computationally practical security improvement for existing industrial embedded systems.

Contents

Abstract	i
1 Introduction	1
1.1 Problem Statement	1
1.2 Project Objectives	1
2 Theoretical Background	2
2.1 The Modbus Protocol	2
2.1.1 Data Model	2
2.2 Evolution to Modbus/TCP	2
3 Threat Landscape for Modbus/TCP	3
3.1 Lack of Authentication	3
3.2 Eavesdropping (Loss of Confidentiality)	3
3.3 Data Injection and Manipulation (Loss of Integrity)	3
3.4 Replay Attacks	3
3.5 Historical Precedent: The Stuxnet Context	3
4 Cryptographic Mitigation: SSH Tunneling	4
4.1 The Mechanics of Secure Shell (SSH)	4
4.2 Local Port Forwarding Architecture	4
5 Project Architecture and Implementation	5
5.1 Component Breakdown	5
5.1.1 The Virtual PLC (Server)	5
5.1.2 The Modbus Client	6
5.1.3 The SSH Tunnel Orchestrator	6
5.2 Environment Automation via Makefile	6
6 Implementation Setup and Key Workarounds	8
6.1 Port 5020 vs Port 502 (Privilege Separation)	8
6.2 Client-Side Data Generation	8
7 Execution Phase I: Plaintext Vulnerability Analysis	9
7.1 Base System Execution	9
7.2 Proving the Eavesdropping Vulnerability	10
7.2.1 Vulnerability Confirmation	10

8	Execution Phase II: Secure Tunnel Mitigation	12
8.1	Establishing the Encrypted Tunnel	12
8.2	Routing Client Traffic through the Tunnel	13
8.3	Proving the Cryptographic Solution	14
8.3.1	Solution Confirmation	14
9	Performance Benchmarking and Feasibility	16
9.1	Methodology	16
9.2	Empirical Results and Analysis	17
9.2.1	Latency Overhead Analysis	17
9.2.2	CPU Utilization Analysis	17
10	Future Trends in Embedded System Security	18
10.1	Protocol Migration	18
10.2	Zero Trust Architecture (ZTA) in OT	18
10.3	Hardware Security Modules (HSMs)	18
10.4	AI-Powered Intrusion Detection Systems (IDS)	18
11	Conclusion	19

List of Figures

1	Clean Architecture mapping of the Application, Security, and Network layers.	5
2	Project Makefile Automation	7
3	Terminal output: Initialization of the Virtual PLC Modbus Server. . . .	9
4	Terminal output: The Modbus Client successfully writing and reading Boiler Temperatures and Motor Speeds.	9
5	Wireshark Capture: The plaintext Boiler Temperature fully exposed in the payload.	10
6	Wireshark Capture: The plaintext Motor Speed fully exposed.	11
7	Option A: Establishing the tunnel utilizing the Native macOS SSH Client via command line.	12
8	Option B: Establishing the tunnel utilizing the cross-platform Python <code>sshtunnel</code> library.	12
9	Terminal output: The Modbus Client seamlessly interacting with the server via the encrypted <code>localhost:2222</code> endpoint.	13
10	Wireshark Capture: The outgoing Modbus write request mathematically obscured within an Encrypted SSH Packet.	14
11	Wireshark Capture: The incoming Modbus read response fully encrypted.	15
12	Terminal output: Execution of the 1000-request benchmarking suite. . . .	16
13	Matplotlib visualization contrasting the Latency (left) and CPU (right) overhead of plaintext vs. encrypted Modbus communication.	17

1 Introduction

Industrial Control Systems (ICS) and SCADA systems basically run the modern world. They control everything from power grids and water treatment plants to manufacturing floors. Historically, these systems were physically isolated from public networks via "air-gapping."

However, Industry 4.0 and the Industrial Internet of Things (IIoT) have changed how these systems operate. Since companies nowadays want real-time data and remote access, the idea of an 'air-gapped' isolated network is mostly a thing of the past. Today, industrial systems are interconnected, bridging Operational Technology (OT) and Information Technology (IT).

Connecting these networks exposes older protocols to modern cyber threats. The most prominent is Modbus. While Modbus offers great simplicity and reliability, it was engineered in 1979 without cybersecurity in mind. As the protocol adapted from serial communication lines to Ethernet (Modbus/TCP), its lack of security controls became a critical liability.

1.1 Problem Statement

Modbus/TCP transmits all control data and sensor readings in absolute plaintext. It requires no authentication to establish a connection, and it provides no cryptographic checksums to verify that a packet has not been altered in transit. If an adversary gains access to the local industrial network, they can easily eavesdrop on sensitive operational data or, far more dangerously, inject malicious control commands directly into the Programmable Logic Controllers (PLCs) governing physical machinery.

1.2 Project Objectives

Our main goal for this project was to explore the vulnerabilities of Modbus/TCP and actually build a practical, affordable way to secure it. Specifically, this project will:

1. Establish a virtual testbed consisting of a Modbus/TCP Client and Server (Virtual PLC).
2. Show the protocol's vulnerability in practice by capturing and analyzing plaintext traffic using Wireshark.
3. Set up a cryptographic wrapper using SSH Tunneling (Port Forwarding) to encapsulate the legacy protocol.
4. Prove the effectiveness of the mitigation by analyzing the encrypted network traffic.
5. Test the performance overhead of the encryption to see if it is practical for resource-constrained embedded systems.

2 Theoretical Background

2.1 The Modbus Protocol

Developed by Modicon (now Schneider Electric) in 1979, Modbus is arguably the most widely used industrial communication protocol in the world. Its success is attributed to its open-source nature, simplicity, and ease of deployment.

Modbus operates on a strict Master/Slave architecture (modernized to Client/Server terminology in Ethernet deployments). In this architecture, only the Master (Client) can initiate communication. The Slaves (Servers/PLCs) remain silent on the network until they are directly polled or commanded by the Master.

2.1.1 Data Model

Modbus interacts with four distinct memory areas within a PLC:

- **Discrete Inputs (Read-Only, 1 bit):** Used for reading binary physical sensors (e.g., limit switches, proximity sensors).
- **Coils (Read/Write, 1 bit):** Used for commanding binary physical actuators (e.g., turning a relay or motor on/off).
- **Input Registers (Read-Only, 16 bits):** Used for reading analog physical sensors (e.g., temperature probes, pressure transducers).
- **Holding Registers (Read/Write, 16 bits):** Used for storing configuration variables or commanding analog actuators (e.g., setting a VFD motor speed).

2.2 Evolution to Modbus/TCP

As industrial networks evolved from specialized serial cables (RS-232/RS-485) to standard Ethernet (TCP/IP), the Modbus protocol was adapted to create Modbus/TCP.

In Modbus/TCP, the traditional serial checksum (CRC) is discarded, relying entirely on the underlying TCP/IP stack for basic transmission error checking. A new header, the Modbus Application Protocol (MBAP) header, is prepended to the Modbus Protocol Data Unit (PDU).

The MBAP header is 7 bytes long and contains:

- **Transaction Identifier (2 bytes):** Used to match responses to requests.
- **Protocol Identifier (2 bytes):** Always 0x0000 for Modbus.
- **Length (2 bytes):** The number of following bytes in the message.
- **Unit Identifier (1 byte):** Used for routing to specific devices on a serial sub-network.

Standard Modbus/TCP operates on **TCP Port 502**.

3 Threat Landscape for Modbus/TCP

The simplicity that makes Modbus/TCP reliable also makes it very insecure. But when you put Modbus/TCP on a connected network today, it creates huge security holes that are easy to exploit.

3.1 Lack of Authentication

Modbus/TCP does not require a username, password, or cryptographic handshake to establish a connection. Any device reaching the IP address and Port 502 of a Modbus PLC can immediately read memory or issue commands. If an attacker breaches the perimeter network, the PLCs are entirely defenseless.

3.2 Eavesdropping (Loss of Confidentiality)

Because data is transmitted in plaintext, an attacker compromising a network switch (e.g., via ARP spoofing) can read all operational data using packet sniffers like Wireshark. In an industrial context, eavesdropping allows attackers to map the physical process, deduce production recipes, and understand system timing, which helps them plan a physical attack.

3.3 Data Injection and Manipulation (Loss of Integrity)

Since the protocol lacks cryptographic signatures or Message Authentication Codes (MACs), an attacker can easily alter packets in transit or forge entirely new packets. For example, an attacker could:

- Send a rogue **Write Single Coil** command to unexpectedly open a physical valve, potentially causing chemical spills.
- Send a rogue **Write Holding Register** command to alter the setpoint of a centrifuge, causing mechanical failure.

3.4 Replay Attacks

Because there are no cryptographic sequence numbers, an attacker can just record a normal command (like "Start Motor") and replay it later to make the PLC do something we didn't ask it to do.

3.5 Historical Precedent: The Stuxnet Context

These are real-world vulnerabilities. The 2010 Stuxnet worm showed the massive damage that compromised PLCs can cause. Stuxnet's payload silently modified the control logic of Siemens PLCs governing uranium centrifuges. Lacking integrity checks, the PLCs blindly accepted the malicious logic, resulting in physical destruction.

4 Cryptographic Mitigation: SSH Tunneling

Most factories can't just throw away all their old PLCs and buy brand new secure hardware. That means we have to find ways to add security on top of the existing network.

This project implements **SSH Tunneling** (SSH Port Forwarding) as a transparent cryptographic wrapper.

4.1 The Mechanics of Secure Shell (SSH)

SSH securely operates network services over unsecured networks, providing three main security features that Modbus/TCP lacks:

1. **Authentication:** Uses public/private key pairs (e.g., RSA or Ed25519) to mathematically prove endpoint identities before data exchange.
2. **Confidentiality (Encryption):** Uses symmetric encryption (e.g., AES-256-GCM) to scramble the payload. Intercepted data appears as random noise without the session key.
3. **Integrity (MAC):** Appends a Hash-based Message Authentication Code (HMAC) to packets. If a bit is altered, MAC verification fails and the packet is dropped, preventing tampering.

4.2 Local Port Forwarding Architecture

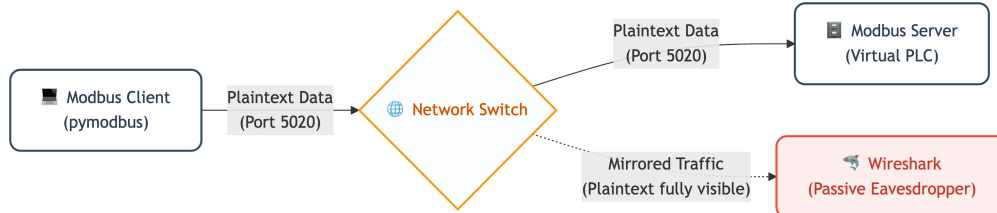
This mitigation intercepts Modbus traffic locally before it reaches the physical network.

- **Step 1: Tunnel Creation:** The SSH Client establishes an encrypted connection to the target SSH Daemon (or edge gateway).
- **Step 2: Local Binding:** The SSH Client opens a local port (e.g., `localhost:2222`).
- **Step 3: Redirection:** The Modbus Client connects to `localhost:2222` instead of the distant PLC.
- **Step 4: Encapsulation:** The SSH client encrypts the Modbus traffic via AES and funnels it through Port 22.
- **Step 5: Decapsulation:** The receiving SSH Daemon decrypts the traffic and forwards raw Modbus bytes to the PLC (Port 5020).

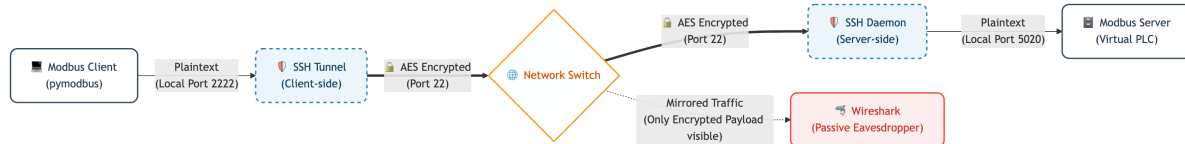
Because encryption occurs at the transport layer, the Modbus application layer remains unaware. No modifications to the Modbus protocol or Python logic are required.

5 Project Architecture and Implementation

The project is structured around the principles of Clean Architecture, separating the Application Layer (the industrial logic) from the Transport and Security Layer (the cryptographic tunnel). The entire environment is written in Python 3 utilizing the `pymodbus` library for industrial simulation and the `sshtunnel` library for cryptographic routing.



(a) Scenario 1: Base System (Vulnerable Plaintext)



(b) Scenario 2: Secure System (SSH Tunneled)

Figure 1: Clean Architecture mapping of the Application, Security, and Network layers.

5.1 Component Breakdown

We ended up splitting the code into three main Python scripts, which are saved in the `src/` folder.

5.1.1 The Virtual PLC (Server)

The `modbus_server.py` script utilizes `StartAsyncTcpServer` from the `pymodbus` library to instantiate a virtual PLC in memory. It allocates blocks of memory (DataBlocks) to represent physical registers.

```
1 from pymodbus.server import StartAsyncTcpServer
2 from pymodbus.datastore import ModbusSequentialDataBlock
3 from pymodbus.datastore import ModbusSlaveContext, ModbusServerContext
4
5 # Initialize memory blocks for the virtual PLC
6 store = ModbusSlaveContext(
7     di=ModbusSequentialDataBlock(0, [0]*100),
8     co=ModbusSequentialDataBlock(0, [0]*100),
9     hr=ModbusSequentialDataBlock(0, [0]*100), # Holding Registers
10    ir=ModbusSequentialDataBlock(0, [0]*100))
11
12 context = ModbusServerContext(slaves=store, single=True)
13
14 # Start listening on the specified port
15 StartAsyncTcpServer(context, address=("0.0.0.0", 5020))
```

Listing 1: Server Initialization Snippet

5.1.2 The Modbus Client

The `modbus_client.py` script acts as the industrial operator or SCADA terminal. It establishes a `ModbusTcpClient` connection. To simulate a dynamic factory environment, it continuously writes changing integer values representing Boiler Temperatures (75-80°C) and Motor Speeds (1450-1500 RPM) to the server's Holding Registers, and subsequently reads them back.

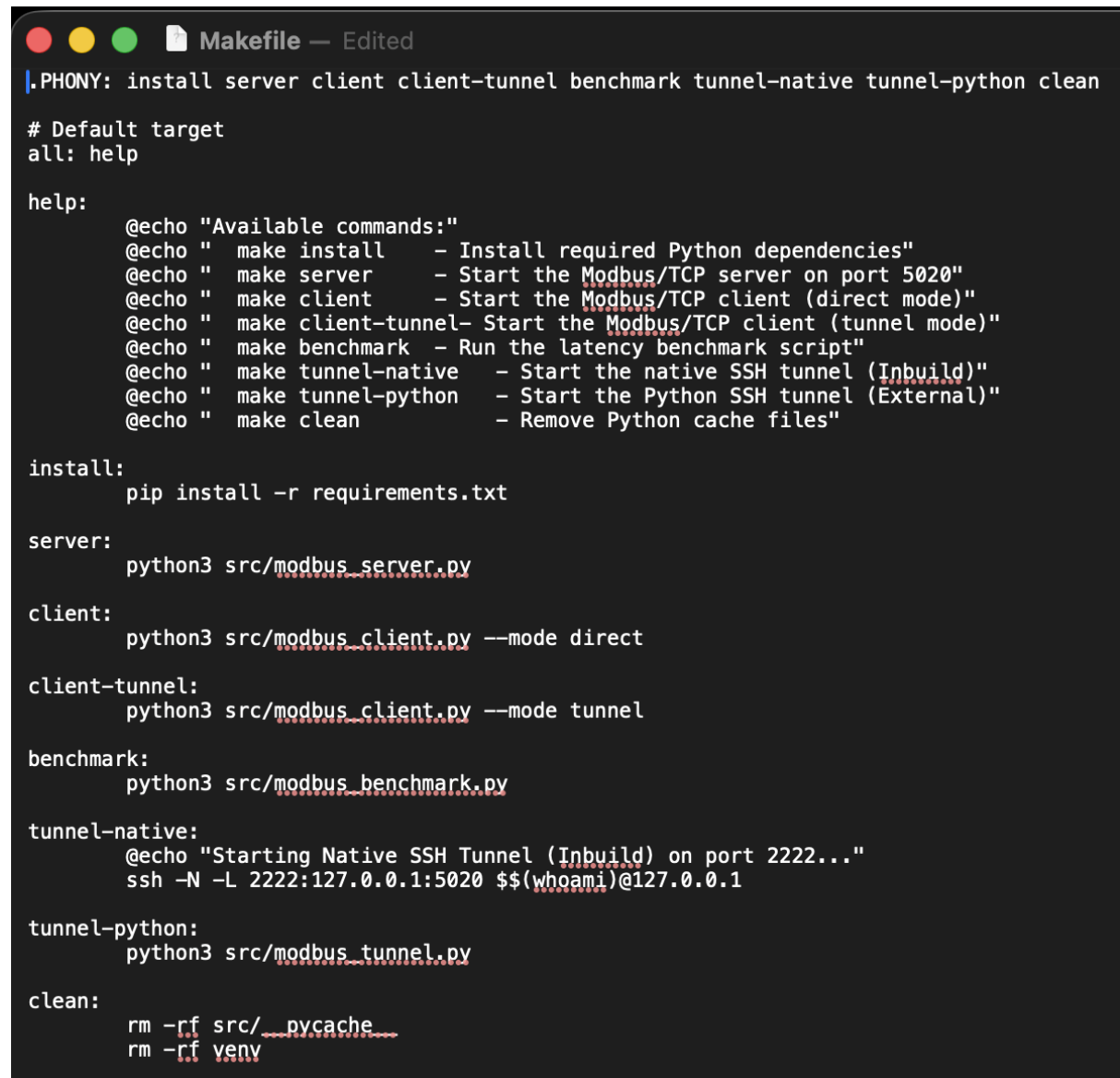
5.1.3 The SSH Tunnel Orchestrator

The `modbus_tunnel.py` script dynamically constructs the cryptographic wrapper. It prompts the user for local system credentials and establishes a Local Port Forward from `localhost:2222` to `127.0.0.1:5020`.

5.2 Environment Automation via Makefile

We wanted to make sure our tests were easy to run and reproduce, so we automated everything using a basic `Makefile`. This abstracts away the underlying Python execution arguments and provides a clean, unified interface for launching the virtual PLC, the industrial client, and the cryptographic tunnels.

This automation is strictly utilized in the subsequent execution phases (Sections 7 and 8) to seamlessly transition the architecture between its native, vulnerable state and its highly secure, tunneled state.



```
Makefile — Edited
.PHONY: install server client client-tunnel benchmark tunnel-native tunnel-python clean

# Default target
all: help

help:
    @echo "Available commands:"
    @echo "  make install    - Install required Python dependencies"
    @echo "  make server     - Start the Modbus/TCP server on port 5020"
    @echo "  make client     - Start the Modbus/TCP client (direct mode)"
    @echo "  make client-tunnel - Start the Modbus/TCP client (tunnel mode)"
    @echo "  make benchmark  - Run the latency benchmark script"
    @echo "  make tunnel-native - Start the native SSH tunnel (Inbuild)"
    @echo "  make tunnel-python - Start the Python SSH tunnel (External)"
    @echo "  make clean      - Remove Python cache files"

install:
    pip install -r requirements.txt

server:
    python3 src/modbus_server.py

client:
    python3 src/modbus_client.py --mode direct

client-tunnel:
    python3 src/modbus_client.py --mode tunnel

benchmark:
    python3 src/modbus_benchmark.py

tunnel-native:
    @echo "Starting Native SSH Tunnel (Inbuild) on port 2222..."
    ssh -N -L 2222:127.0.0.1:5020 $(whoami)@127.0.0.1

tunnel-python:
    python3 src/modbus_tunnel.py

clean:
    rm -rf src/.pycache
    rm -rf venv
```

Figure 2: Project Makefile Automation

6 Implementation Setup and Key Workarounds

To simulate an industrial environment on standard computer hardware, we had to implement two main technical workarounds.

6.1 Port 5020 vs Port 502 (Privilege Separation)

Standard Modbus hardware inherently operates on TCP Port 502. However, the simulation environment was executed on macOS (a UNIX-based operating system). A basic security feature of UNIX is that standard user accounts cannot bind services to ports under 1024.

Attempting to run a Modbus server on Port 502 would require executing the Python script with `sudo` (root privileges). Executing network-facing scripts as root poses a massive security risk to the host machine.

Solution: As a security best practice, the virtual PLC was designed to listen on **Port 5020**. This avoids the need for root access while still preserving the functionality of the TCP/IP protocol stack and the Modbus Application Protocol.

6.2 Client-Side Data Generation

In a physical factory, the PLC (Server) is hardwired directly to physical sensors (e.g., thermocouples, tachometers). The PLC natively generates the data and populates its own memory registers, waiting for a SCADA client to read it.

For this software testbed, the virtual PLC is effectively a "blank brain." It contains memory registers, but no background threads to automatically update them with simulated physics. Writing complex background polling threads for the server would overcomplicate the codebase and distract from the core security focus.

Solution: Instead of the Server generating data, the **Client** was programmed to act as the environment simulator. To keep things simple, we just programmed the client to generate the random Boiler Temperatures and Motor Speeds instead of writing a background thread for the server.

- The client initiates a **Write** command to push the data into the server's memory.
- Immediately following, the client initiates a **Read** command to pull the data back out.

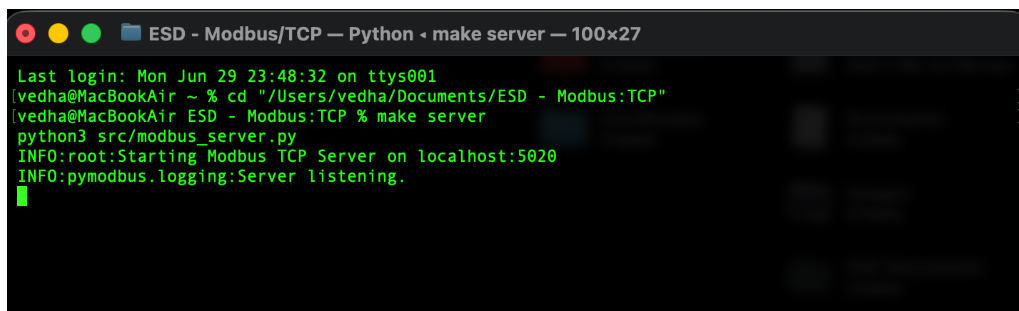
The Benefit: This setup made it much easier for us to analyze the network. Because the client was writing and reading data, we were able to capture both incoming and outgoing traffic in Wireshark and prove it was encrypted in both directions.

7 Execution Phase I: Plaintext Vulnerability Analysis

For the first part of the project, we just needed to establish a baseline to prove that standard Modbus/TCP is completely insecure out of the box.

7.1 Base System Execution

The virtual PLC server is instantiated, and the Modbus client is executed in "direct" mode, bypassing any encryption and connecting directly to the server on Port 5020.

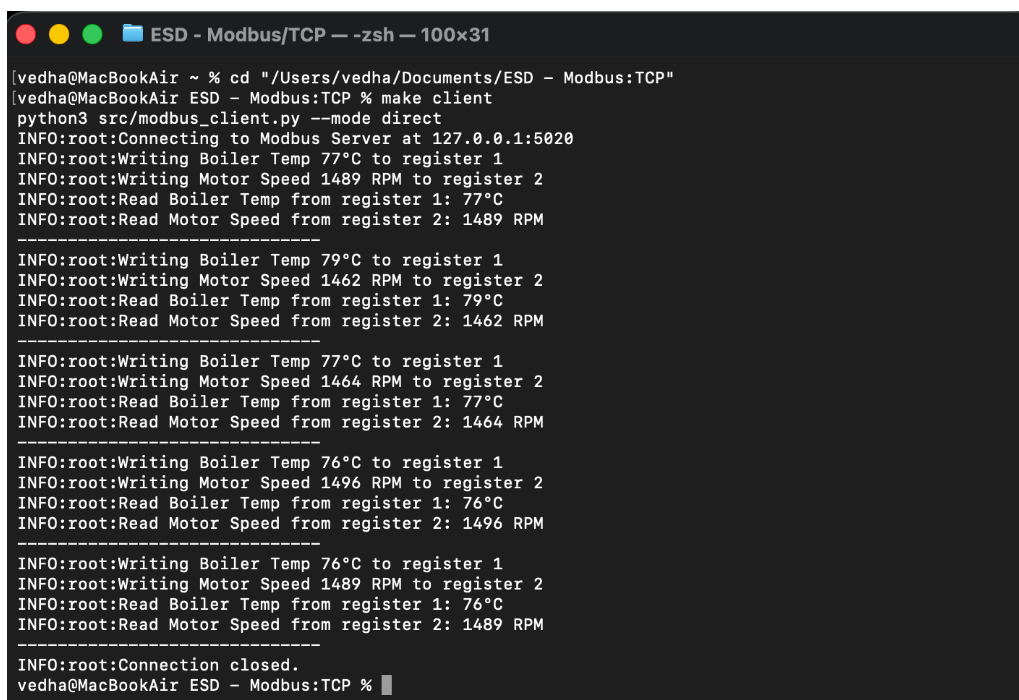


```

ESD - Modbus/TCP — Python — make server — 100x27
Last login: Mon Jun 29 23:48:32 on ttys001
[vedha@MacBookAir ~ % cd "/Users/vedha/Documents/ESD - Modbus:TCP"
[vedha@MacBookAir ESD - Modbus:TCP % make server
python3 src/modbus_server.py
INFO:root:Starting Modbus TCP Server on localhost:5020
INFO:pymodbus.logging:Server listening.

```

Figure 3: Terminal output: Initialization of the Virtual PLC Modbus Server.



```

ESD - Modbus/TCP — -zsh — 100x31
[vedha@MacBookAir ~ % cd "/Users/vedha/Documents/ESD - Modbus:TCP"
[vedha@MacBookAir ESD - Modbus:TCP % make client
python3 src/modbus_client.py --mode direct
INFO:root:Connecting to Modbus Server at 127.0.0.1:5020
INFO:root:Writing Boiler Temp 77°C to register 1
INFO:root:Writing Motor Speed 1489 RPM to register 2
INFO:root:Read Boiler Temp from register 1: 77°C
INFO:root:Read Motor Speed from register 2: 1489 RPM
-----
INFO:root:Writing Boiler Temp 79°C to register 1
INFO:root:Writing Motor Speed 1462 RPM to register 2
INFO:root:Read Boiler Temp from register 1: 79°C
INFO:root:Read Motor Speed from register 2: 1462 RPM
-----
INFO:root:Writing Boiler Temp 77°C to register 1
INFO:root:Writing Motor Speed 1464 RPM to register 2
INFO:root:Read Boiler Temp from register 1: 77°C
INFO:root:Read Motor Speed from register 2: 1464 RPM
-----
INFO:root:Writing Boiler Temp 76°C to register 1
INFO:root:Writing Motor Speed 1496 RPM to register 2
INFO:root:Read Boiler Temp from register 1: 76°C
INFO:root:Read Motor Speed from register 2: 1496 RPM
-----
INFO:root:Writing Boiler Temp 76°C to register 1
INFO:root:Writing Motor Speed 1489 RPM to register 2
INFO:root:Read Boiler Temp from register 1: 76°C
INFO:root:Read Motor Speed from register 2: 1489 RPM
-----
INFO:root:Connection closed.
vedha@MacBookAir ESD - Modbus:TCP %

```

Figure 4: Terminal output: The Modbus Client successfully writing and reading Boiler Temperatures and Motor Speeds.

From the application's perspective, the operations are successful. The Client reports successfully writing a Boiler Temperature to Register 1, and a Motor Speed to Register 2.

7.2 Proving the Eavesdropping Vulnerability

While the application works normally, the network traffic is completely exposed. Wireshark, a prominent network protocol analyzer, was attached to the loopback interface (lo0) to silently capture all packets traversing Port 5020.

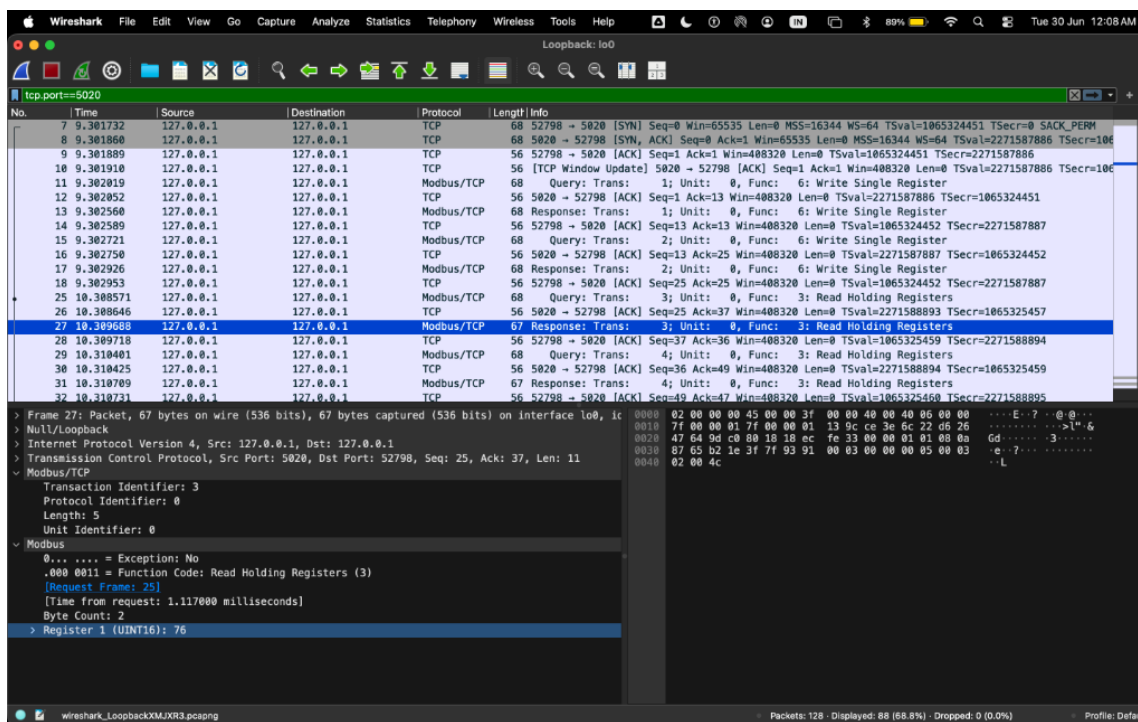


Figure 5: Wireshark Capture: The plaintext Boiler Temperature fully exposed in the payload.

7.2.1 Vulnerability Confirmation

As you can see in Figures 5 and 6, Wireshark has no problem reading the Modbus/TCP packets. It easily picks up the Function Code (Read Holding Registers (3)) and shows the exact numbers sent in the payload.

This confirms the main vulnerability: an attacker possessing physical or logical access to the network switch can monitor the entire industrial process without triggering any alarms.

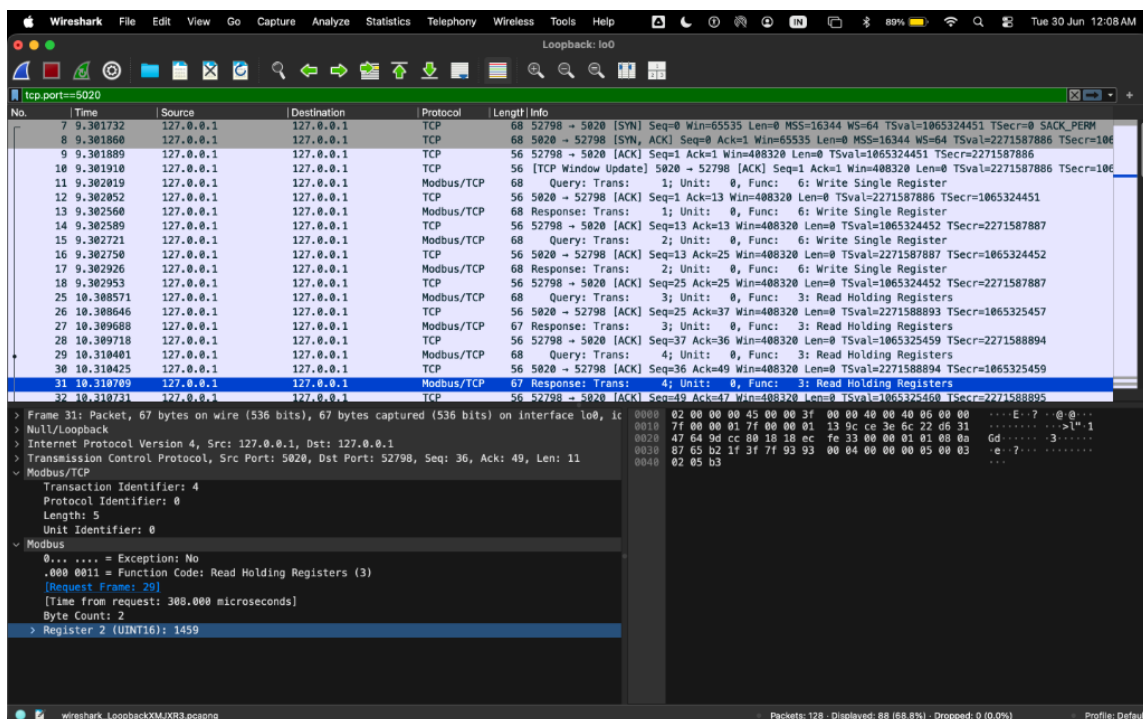


Figure 6: Wireshark Capture: The plaintext Motor Speed fully exposed.

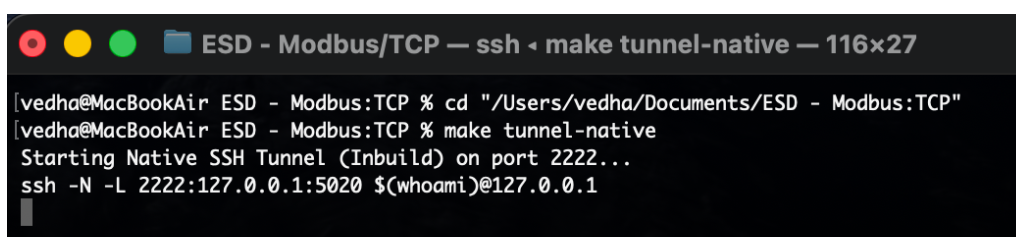
8 Execution Phase II: Secure Tunnel Mitigation

To fix the problem, we set up a secure SSH tunnel and routed all the traffic through it. An SSH tunnel is established to encapsulate and encrypt the industrial traffic.

macOS System Requirement: In order for the local machine to act as the receiving end of the SSH tunnel, the local SSH Daemon (`sshd`) must be active. On macOS, this requires navigating to System Settings > General > Sharing and explicitly enabling the "Remote Login" feature.

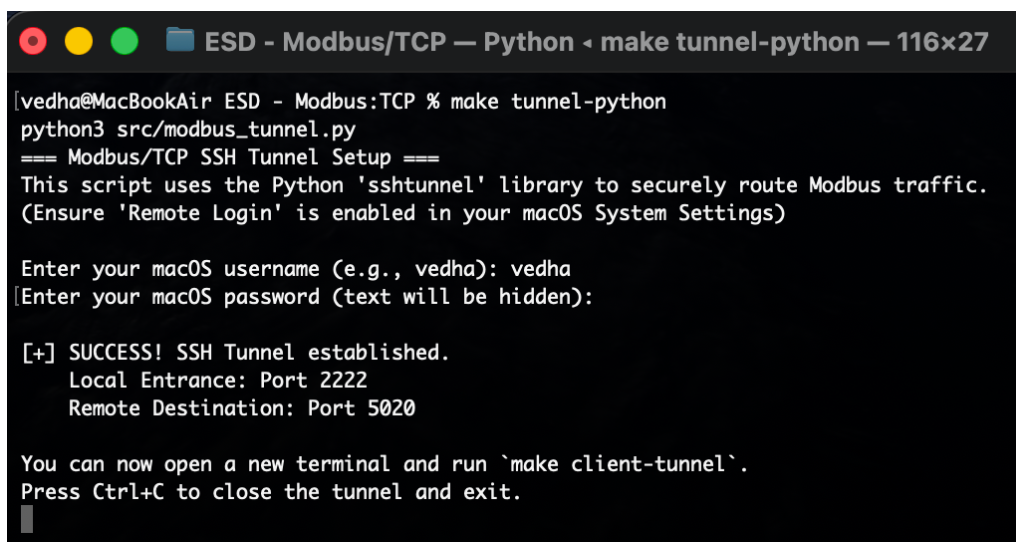
8.1 Establishing the Encrypted Tunnel

The project facilitates two distinct methods for establishing the SSH tunnel.



```
ESD - Modbus/TCP — ssh ◀ make tunnel-native — 116x27
[vedha@MacBookAir ESD - Modbus:TCP % cd "/Users/vedha/Documents/ESD - Modbus:TCP"
[vedha@MacBookAir ESD - Modbus:TCP % make tunnel-native
Starting Native SSH Tunnel (Inbuild) on port 2222...
ssh -N -L 2222:127.0.0.1:5020 $(whoami)@127.0.0.1
```

Figure 7: Option A: Establishing the tunnel utilizing the Native macOS SSH Client via command line.



```
ESD - Modbus/TCP — Python ◀ make tunnel-python — 116x27
[vedha@MacBookAir ESD - Modbus:TCP % make tunnel-python
python3 src/modbus_tunnel.py
=== Modbus/TCP SSH Tunnel Setup ===
This script uses the Python 'sshtunnel' library to securely route Modbus traffic.
(Ensure 'Remote Login' is enabled in your macOS System Settings)

Enter your macOS username (e.g., vedha): vedha
[Enter your macOS password (text will be hidden):

[+] SUCCESS! SSH Tunnel established.
    Local Entrance: Port 2222
    Remote Destination: Port 5020

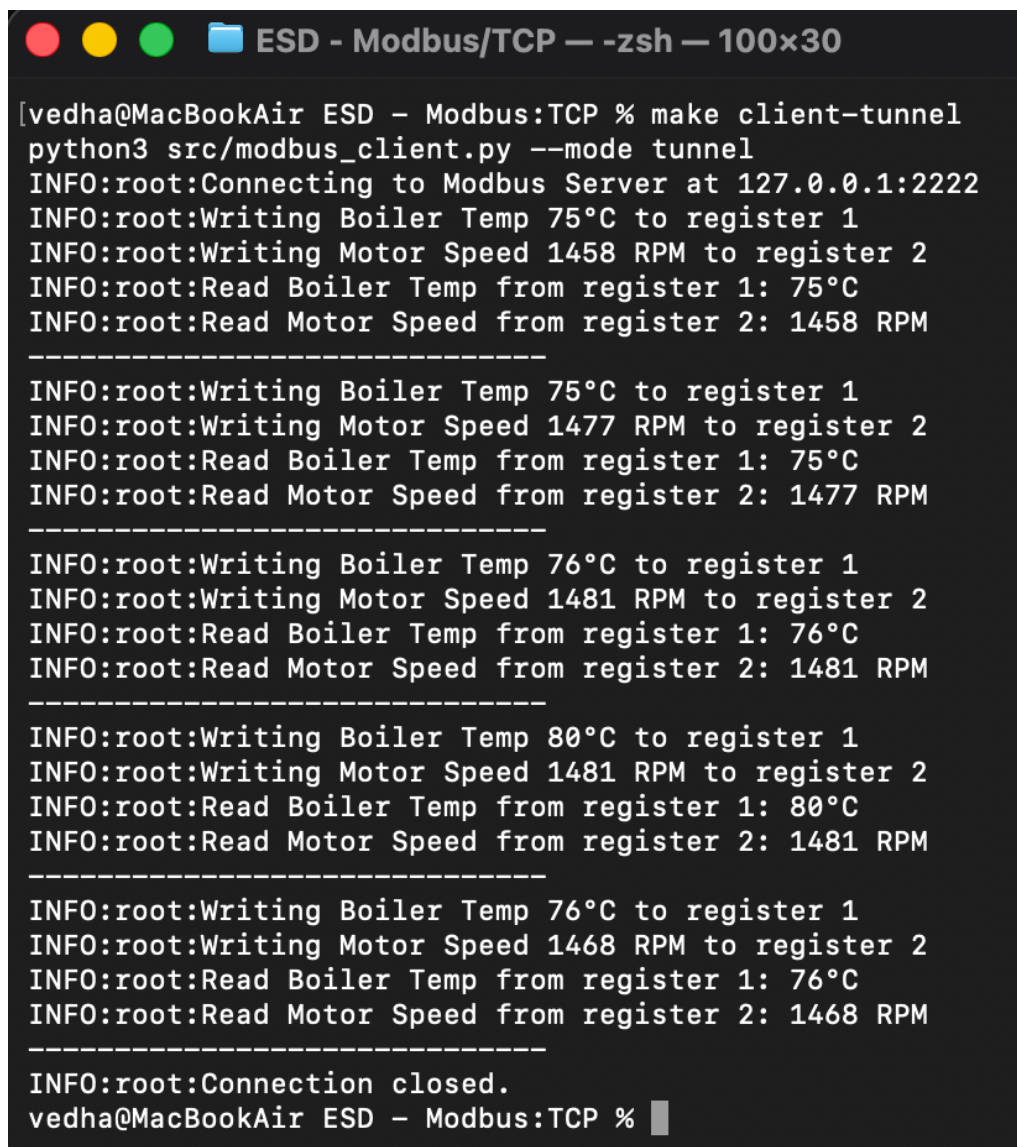
You can now open a new terminal and run `make client-tunnel`.
Press Ctrl+C to close the tunnel and exit.
```

Figure 8: Option B: Establishing the tunnel utilizing the cross-platform Python `sshtunnel` library.

Both methods achieve identical outcomes: a secure, local listening port is established on `localhost:2222`, which transparently encrypts and forwards all incoming data to the actual Modbus server on Port 5020.

8.2 Routing Client Traffic through the Tunnel

With the cryptographic wrapper active, the Modbus client is executed in "tunnel" mode. The application code is dynamically re-configured to point its TCP socket to the new local endpoint (`localhost:2222`) rather than directly to the server.



```
[vedha@MacBookAir ESD - Modbus:TCP % make client-tunnel
python3 src/modbus_client.py --mode tunnel
INFO:root:Connecting to Modbus Server at 127.0.0.1:2222
INFO:root:Writing Boiler Temp 75°C to register 1
INFO:root:Writing Motor Speed 1458 RPM to register 2
INFO:root:Read Boiler Temp from register 1: 75°C
INFO:root:Read Motor Speed from register 2: 1458 RPM
-----
INFO:root:Writing Boiler Temp 75°C to register 1
INFO:root:Writing Motor Speed 1477 RPM to register 2
INFO:root:Read Boiler Temp from register 1: 75°C
INFO:root:Read Motor Speed from register 2: 1477 RPM
-----
INFO:root:Writing Boiler Temp 76°C to register 1
INFO:root:Writing Motor Speed 1481 RPM to register 2
INFO:root:Read Boiler Temp from register 1: 76°C
INFO:root:Read Motor Speed from register 2: 1481 RPM
-----
INFO:root:Writing Boiler Temp 80°C to register 1
INFO:root:Writing Motor Speed 1481 RPM to register 2
INFO:root:Read Boiler Temp from register 1: 80°C
INFO:root:Read Motor Speed from register 2: 1481 RPM
-----
INFO:root:Writing Boiler Temp 76°C to register 1
INFO:root:Writing Motor Speed 1468 RPM to register 2
INFO:root:Read Boiler Temp from register 1: 76°C
INFO:root:Read Motor Speed from register 2: 1468 RPM
-----
INFO:root:Connection closed.
vedha@MacBookAir ESD - Modbus:TCP %
```

Figure 9: Terminal output: The Modbus Client seamlessly interacting with the server via the encrypted `localhost:2222` endpoint.

As evidenced by Figure 9, the application layer is not affected by the transport layer modifications. The read and write operations for Boiler Temperature and Motor Speed execute flawlessly.

8.3 Proving the Cryptographic Solution

To verify the effectiveness of the mitigation, Wireshark is once again utilized. However, because the traffic is now routed through the SSH protocol, the capture filter is adjusted to monitor **TCP Port 22**.

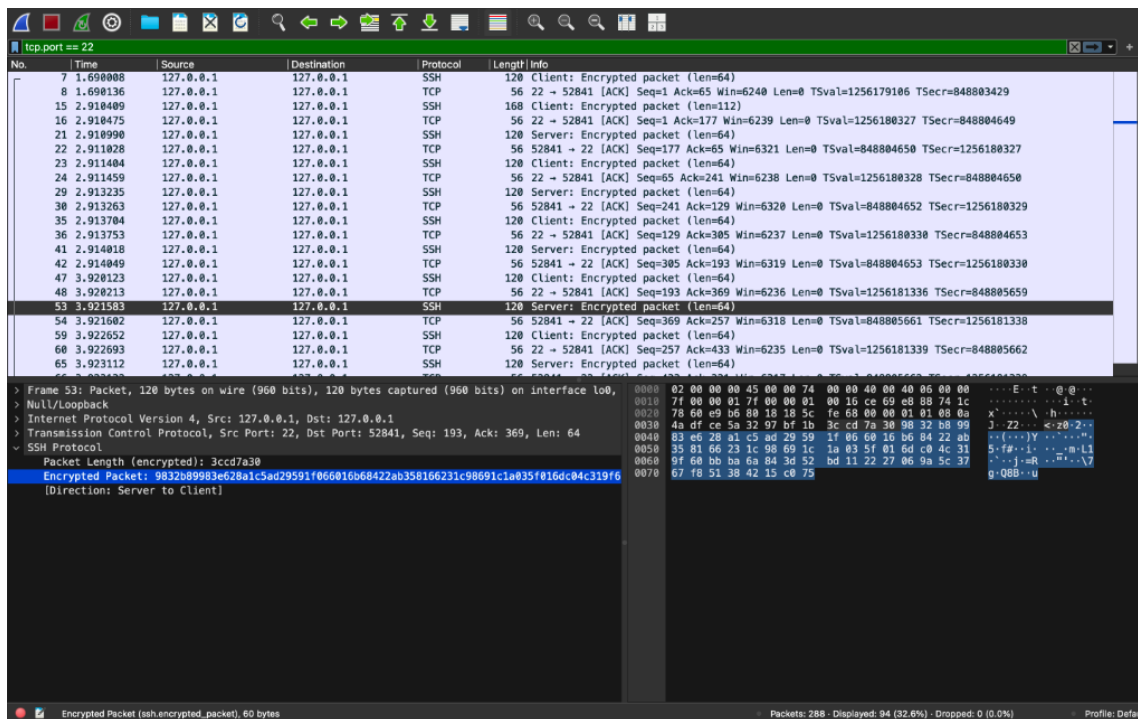


Figure 10: Wireshark Capture: The outgoing Modbus write request mathematically obscured within an Encrypted SSH Packet.

8.3.1 Solution Confirmation

The difference between Phase I and Phase II is clear. As shown in Figures 10 and 11, Wireshark cannot read the contents of the packets.

The packets are strictly identified as SSHv2 protocol packets. The payload section, which previously displayed the plaintext industrial data, is now categorized solely as an "Encrypted Packet." The raw bytes are a mathematically random sequence generated by the AES cipher.

The implementation has successfully achieved:

- **Confidentiality:** The attacker can no longer read the Boiler Temperature or Motor Speed.
- **Integrity:** The HMAC prevents the attacker from blindly injecting altered bits to disrupt the physical process.

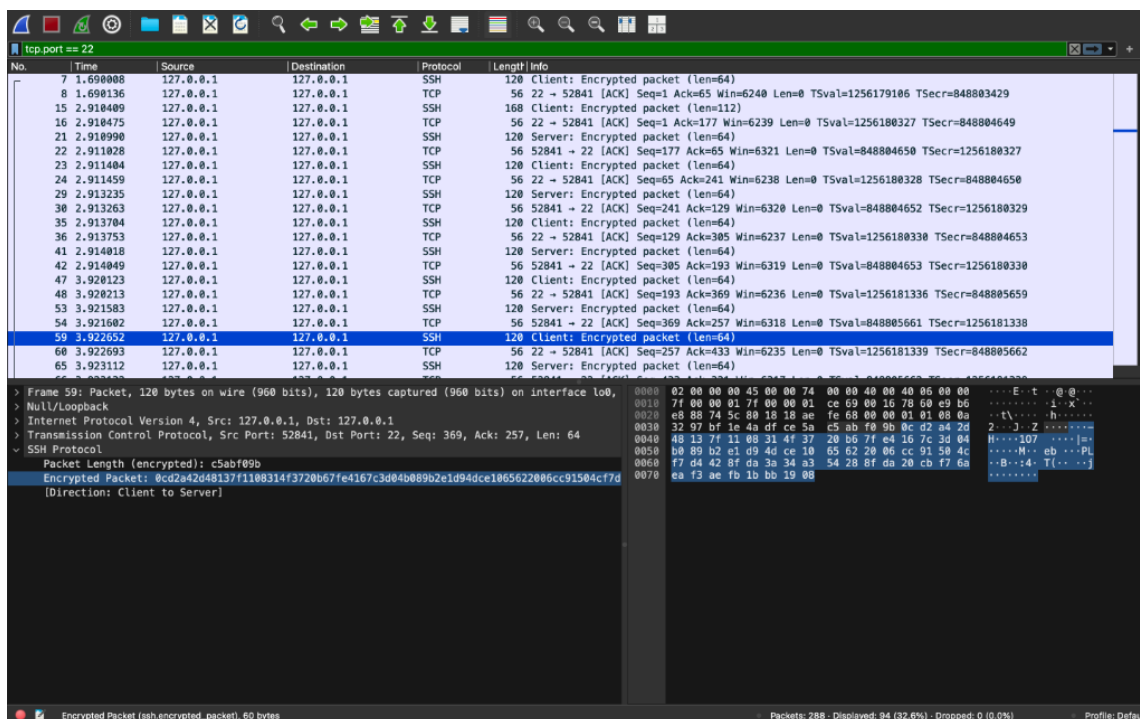


Figure 11: Wireshark Capture: The incoming Modbus read response fully encrypted.

9 Performance Benchmarking and Feasibility

While SSH tunneling provides good security, we still need to evaluate the operational cost. Embedded systems in industrial environments often operate with strictly constrained CPU resources and hard real-time latency requirements. Introducing AES encryption math and HMAC hashing at the transport layer inherently introduces computational overhead.

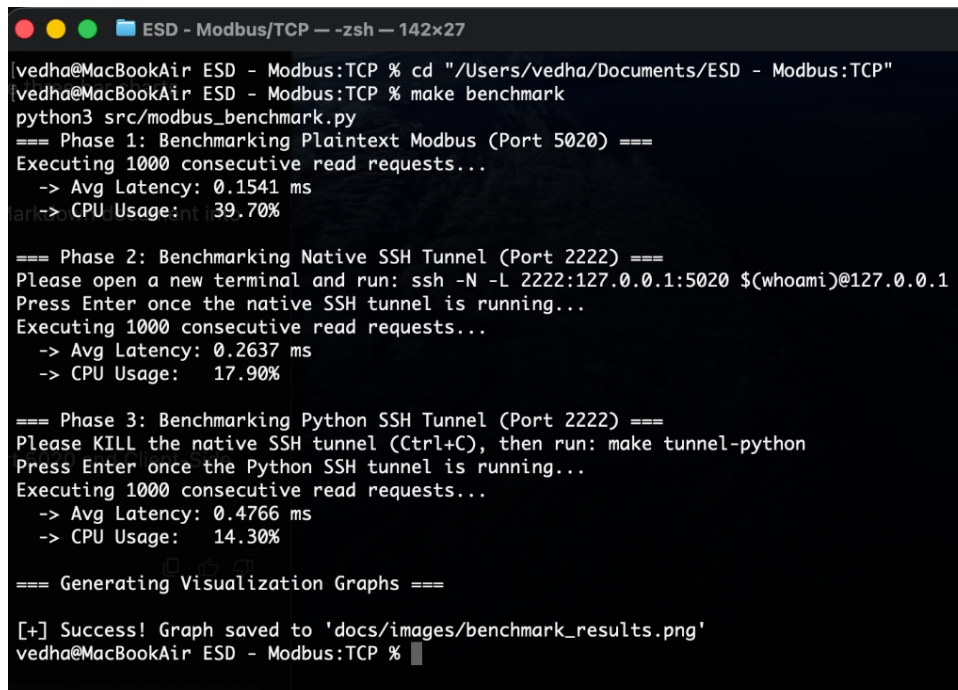
To test this feasibility, we built an automated Python benchmarking suite (`modbus_benchmark.py`) was engineered.

9.1 Methodology

To get an idea of the network overhead, our benchmark script basically spams **1,000 Modbus read requests** as fast as possible across three different setups:

1. **Phase 1: Plaintext Baseline.** Requests are sent directly to Port 5020.
2. **Phase 2: Native SSH Tunnel.** Requests are routed through the macOS native OpenSSH client wrapper.
3. **Phase 3: Python SSH Tunnel.** Requests are routed through the `sshtunnel` library wrapper.

For each setup, we used the Python `time` module to measure latency down to the microsecond, and the `psutil` library to check the CPU usage.



```
ESD - Modbus/TCP — -zsh — 142x27
vedha@MacBookAir ESD - Modbus:TCP % cd "/Users/vedha/Documents/ESD - Modbus:TCP"
vedha@MacBookAir ESD - Modbus:TCP % make benchmark
python3 src/modbus_benchmark.py
=== Phase 1: Benchmarking Plaintext Modbus (Port 5020) ===
Executing 1000 consecutive read requests...
-> Avg Latency: 0.1541 ms
-> CPU Usage: 39.70%

=== Phase 2: Benchmarking Native SSH Tunnel (Port 2222) ===
Please open a new terminal and run: ssh -N -L 2222:127.0.0.1:5020 $(whoami)@127.0.0.1
Press Enter once the native SSH tunnel is running...
Executing 1000 consecutive read requests...
-> Avg Latency: 0.2637 ms
-> CPU Usage: 17.90%

=== Phase 3: Benchmarking Python SSH Tunnel (Port 2222) ===
Please KILL the native SSH tunnel (Ctrl+C), then run: make tunnel-python
Press Enter once the Python SSH tunnel is running...
Executing 1000 consecutive read requests...
-> Avg Latency: 0.4766 ms
-> CPU Usage: 14.30%

=== Generating Visualization Graphs ===

[+] Success! Graph saved to 'docs/images/benchmark_results.png'
vedha@MacBookAir ESD - Modbus:TCP %
```

Figure 12: Terminal output: Execution of the 1000-request benchmarking suite.

9.2 Empirical Results and Analysis

Upon completion, the benchmarking suite leverages the `matplotlib` library to generate comparative bar charts visualizing the latency and CPU utilization deltas.

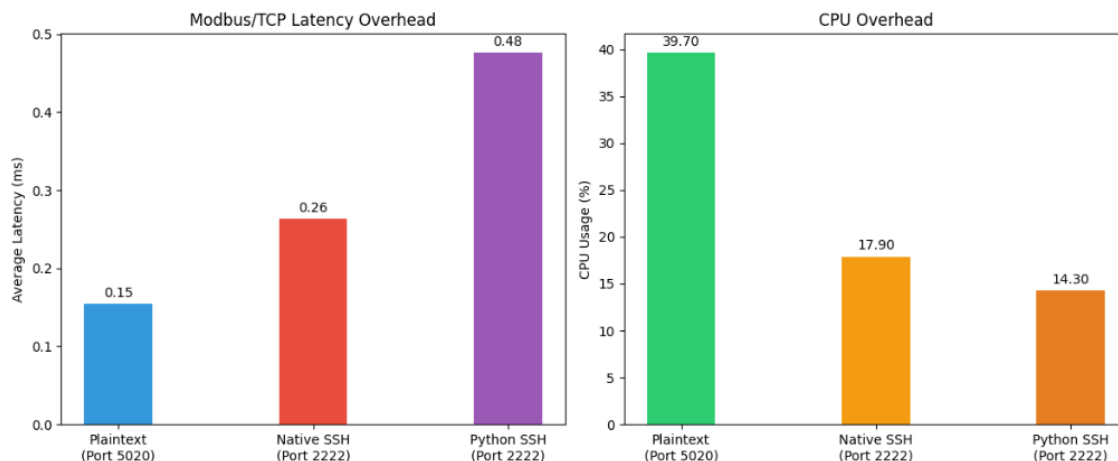


Figure 13: Matplotlib visualization contrasting the Latency (left) and CPU (right) overhead of plaintext vs. encrypted Modbus communication.

9.2.1 Latency Overhead Analysis

The plaintext baseline averaged **0.15 ms** latency per request. Native SSH Tunnel routing increased latency to **0.26 ms**. Python SSH Tunnel routing increased latency to **0.48 ms**.

Conclusion on Latency: While encryption increases response times, values remain under one millisecond. Standard SCADA systems typically poll PLCs at 100-1000 ms intervals, making this sub-millisecond overhead negligible. Data proves SSH tunneling respects timing constraints of standard control loops, though it remains unsuitable for high-speed robotic motion (which utilizes EtherCAT or PROFINET IRT).

9.2.2 CPU Utilization Analysis

CPU metrics must be interpreted concerning a multi-core consumer processor running background tasks. Interestingly, the benchmark recorded higher global CPU utilization during the plaintext phase (39.70%) than the tunneled phases (17.90% and 14.30%). This difference is likely caused by Python's Global Interpreter Lock (GIL) and scheduling inefficiencies during 1000 unthrottled TCP connections, whereas SSH manages the TCP stack more efficiently at the OS level.

Conclusion on CPU: Modern embedded edge gateways (e.g., ARM Cortex-A processors) feature hardware-accelerated cryptographic instructions (like ARMv8 Crypto Extensions), executing AES-GCM encryption natively in silicon. Therefore, the computational footprint of an SSH tunnel on modern embedded hardware is highly manageable, ensuring viable deployment.

10 Future Trends in Embedded System Security

While SSH tunneling is a great temporary fix for legacy protocols, the industry is shifting towards "secure-by-design" architectures. Future embedded security will rely on natively robust protocols and hardware trust.

10.1 Protocol Migration

Instead of adding wrappers to plaintext protocols, networks are adopting standards with built-in security.

- **OPC UA:** This is becoming the standard for industrial interoperability. It builds X.509 authentication and TLS encryption directly into the protocol.
- **MQTT:** For distributed sensor networks, MQTT offers a lightweight publish/subscribe model with native TLS support.

10.2 Zero Trust Architecture (ZTA) in OT

The old "castle-and-moat" defense model—where internal traffic is automatically trusted—is obsolete. Today's standard is Zero Trust: "Never Trust, Always Verify." Even isolated PLCs shouldn't blindly accept commands. Future devices will require identity verification for every transaction, no matter where it comes from.

10.3 Hardware Security Modules (HSMs)

Software encryption stores private keys on the file system, meaning an attacker with physical access could steal them. To fix this, future systems will use hardware-backed security like Trusted Platform Modules (TPMs). These isolate cryptographic keys in tamper-proof chips, keeping them safe even if the OS is compromised.

10.4 AI-Powered Intrusion Detection Systems (IDS)

Because industrial networks generate massive amounts of data, humans can't analyze every packet manually. Instead, machine learning algorithms run directly on network switches to monitor behavior. For example, if an AI learns that a specific terminal only reads data, it will instantly block any sudden write commands from that terminal—even if the traffic comes through a secure SSH tunnel.

11 Conclusion

Connecting operational technology to the internet has transformed industrial environments, making them more efficient and easier to manage remotely. However, this connectivity has also exposed older systems to modern cyber threats.

This project achieved its objectives by clearly demonstrating the vulnerabilities of the Modbus/TCP protocol. By looking directly at the network traffic, we proved that Modbus/TCP sends important control commands and sensor data in completely plain text. It has absolutely no built-in defense against spying or data injection.

To remediate this vulnerability, an architectural redesign utilizing SSH tunneling was successfully implemented. By acting as a transparent cryptographic wrapper, SSH added Authentication, AES encryption for Confidentiality, and HMAC validation for Integrity to the legacy environment. Subsequent deep packet inspection confirmed that the Modbus payload was rendered mathematically opaque to network observers.

Finally, the automated benchmarking suite showed that the performance cost of this security addition is small. The sub-millisecond latency overhead and manageable CPU utilization demonstrate that SSH tunneling is a highly feasible, pragmatic, and cost-effective mitigation strategy for resource-constrained embedded systems.

While the industry is eventually moving toward natively secure protocols like OPC UA, legacy Modbus/TCP infrastructure will likely remain in use for many years. In the meantime, using encryption wrappers like SSH or IPSec VPNs is a practical necessity to protect physical infrastructure from cyber attacks.

References

- [1] Modbus Organization, *MODBUS Application Protocol Specification V1.1b3*, 2006. [Online]. Available: http://www.modbus.org/docs/Modbus_Application_Protocol_V1_1b3.pdf
- [2] T. Ylonen and C. Lonvick, *The Secure Shell (SSH) Protocol Architecture*, RFC 4251, Jan. 2006.
- [3] R. Langner, *Stuxnet: Dissecting a Cyberwarfare Weapon*, IEEE Security & Privacy, vol. 9, no. 3, pp. 49-51, May-June 2011.
- [4] R. R. G., *Pymodbus: A full modbus protocol written in python*, 2023. [Online]. Available: <https://github.com/pymodbus-dev/pymodbus>
- [5] K. Stouffer, V. Pillitteri, S. Lightman, M. Abrams, and A. Hahn, *Guide to Industrial Control Systems (ICS) Security*, NIST Special Publication 800-82 Rev. 2, May 2015.